

Moving Tech AI Website

Frontend documentation movingTechAIFrontend

Version	1.0
Last Updated	04/29/2026

1. Overview

The Moving Tech AI Frontend is a website built using React.js as the base framework and uses NPM libraries for supplemental functions. The application is in a read only state for public visitors, because all data originates from the Firebase database which can't be edited from the public facing website.

2. Repository Structure

The source code for the frontend components sits within the src/ directory on the movingTechAIFrontend folder on the repository. The files listed below help cover the complete front facing website:

File / Path	Description
Src/Index.js	Entry point, this is what mounts the react app into the DOM root
Src/App.js	This is the root component which is fed to the index.js. This defines the BrowserRouter and other mappings
Src/pages/landingPage.jsx	Full landing page with stats, preview cards, and more
Src/pages/directoryPage.jsx	Page for searching through the available companies listed on the website. This full search, filter, pagination, and sorting functionality
Src/pages/softwareDetailsPage.jsx	Individual page for each software profile and their feature matrix to be displayed
Src/api/client.js	The single source of truth for HTTP calls
Src/hooks/useAPI.js	useApi and useDebounce are custom hooks we made
Src/styles/global.css	Styling for overall css variables, layouts, and styles
Src/styles/components.css	Component scoped styling

Note: Each of the main pages have subcomponents within the components src/components folder.

3. Routing Architecture

All Routing is handled by the NPM library React Router using BrowserRouter. The routes are defined in the App.js.

Route Table:

Path	File	Description
/	landingPage.jsx	This is the landing page for any users who want to use the website
/directory	directoryPage.jsx	This is the page hosting the search, filter, and sort tools for the available companies
/software/id	softwareDetailPage.jsx	Full profile for each company's product which is identified to their slug in the database

Note: the id parameter for the software profile page is mapped to the firebase's document slug (example: smartmoving, articeck, etc)

4. API Information

4.1 Client.js

All communication with the backend is done via HTTP requests via the client.js file. This file contains four functions which are exported for use. Each of the four functions calls the async internal request() helper function which constructs the full HTTP request URL based on the environment variables, if the env variable is not there the code defaults the URL to a localhost URL.

4.2 Environment Config

The base URL is read from the REACT_APP_API_URL environment variable. For production use, ensure in the Render service the environment variable is set and pointing to the backend WebService for the backend.

4.3 API Functions

Function	Endpoint	Description
searchSoftware(q, page, pageSize, sortBy)	GET /api/software/search	Full text search by name, description, or keywords. This returns paginated results.
filterSoftware(filters, page, pageSize, sortBy)	GET /api/software/filter	Filter by typology, market, install type, and feature flags. Also returns paginated results.
getFilterOptions()	GET /api/software/options	This returns available values for all filter dropdowns. Called once on DirectoryPage mount.
getSoftwareById(id)	GET /api/software/:id	This will get a single software profile by its Firebase slug. Used on the detail page for direct URL access.

4.3 Search Query Parameters

searchSoftware and filterSoftware both accept the following parameters:

Parameter	Default	Description
Page	1	Page number
Pagesize	20	Amount of results per page
Sort_by	Verified_first	This is the sorting options, it can be: 1. Verified_first, 2. Unverified_first, 3. Name_asc, 4. Name_desc

Note: filterSoftware additionally appends one parameter per active filter key. An Array valued filters (typology, best_for, install, language, status) are added multiple times with the same key name using URLSearchParams.append().

4.4 Custom Hooks, useAPI

This custom hook is a generic data fetching hook which wraps the async function and returns {data, loading, error, refetch}. This uses a mountedRef to safeguard against the state getting set on unmounted components which helps prevent memory leaks during user navigation.

Note: The directory page does not use the useAPI hook directly. Instead, it manages its own state with the useState and useCallback functions to help control over calling the triggers.

4.5 Custom Hooks, useDebounce

This custom hook delays updates by a configurable amount of milliseconds, this is used in the directory page to prevent unnecessary API calls on each key stroke. The value returned is what triggers the fetchResults useCallback rather than the raw search string.

5. Styling Guide

This application uses vanilla CSS design system, with the styles split across two different files:

5.1 Global.css

This contains every style that applied at a page scope or higher:

1. Custom properties are saved in :root variables which are single source of truth for colors, spacing, shadows, and typography styling.
2. Universal styling like box sizing, margin, paddings.
3. Body defaults like the font, background, line heights, and more
4. Shared button variations
5. Full layouts for each page
6. Sort control and results header styling

5.2 Components.css

This file contains styles which are scoped out to the reuseable UI components like the filterPanel, softwareCard, feature badges, etc. The classes follow a BEM naming convention like the page level classes

6. Frontend Mapping to Database

The frontend takes in the data from the backend's API as a JSON object, below is the table mapping the key frontend fields to the Firebase paths documented in the E2E document:

Frontend Field	Firestore Path	Used In
item.id	Document ID (slug)	React key, URL param, getSoftwareById()
item.name	name	Card title, detail header
item.description	description	Card summary, detail header
item.website	website	Detail page external link
item.typology	software_typology	Card badge, filter, detail typology badge
item.best_for	target_market (joined string)	Overview card, Total Market
item.install	deployment_types (joined string)	Overview card, Deployment
item.language	languages (joined string)	Overview card, Languages
item.keywords	keywords (comma-separated string)	Overview card, search indexing
item.status	status.review_status	Status badge (verified / unverified)
item.features.lead_mgmt	features.sales.lead_management	Feature matrix, Sales section
item.features.surveying	features.sales.surveying	Feature matrix, Sales section

Frontend Field	Firestore Path	Used In
item.features.quoting	features.sales.quoting	Feature matrix, Sales section
item.features.job_booking	features.move_management.job_booking	Feature matrix, Move Mgmt section
item.features.move_mgmt	features.move_management.move_mgmt	Feature matrix, Move Mgmt section
item.features.dispatch	features.operations.dispatch	Feature matrix, Operations section
item.features.shipments	features.operations.shipment	Feature matrix, Operations section
item.features.crew_app	features.operations.crew_app	Feature matrix, Operations section
item.features.storage	features.operations.storage	Feature matrix, Operations section
item.features.ar	features.erp.accounts_receivable	Feature matrix, ERP section
item.features.ap	features.erp.accounts_payable	Feature matrix, ERP section
item.features.hr	features.erp.hr	Feature matrix, ERP section

Note: The backend API will normalize the array fields into a comma separated string before sending it to the frontend. The frontend will then split these strings back into arrays as needed.

7. Filter and Search System

7.1 Filter State

The INITIAL_FILTERS constant in the directoryPage.jsx defines the default filter state object as such:

```
{
  typology: [], best_for: [], install: [], language: [], status: [], lead_mgmt: null, dispatch: null,
  crew_app: null, storage: null, surveying: null
}
```

Array fields can accept multi select values. The Boolean fields can be null (no filter), true (must use filter), or false (don't use filter).

7.2 Search vs. Filter Mode

The directory works in one of two mutually exclusive modes:

1. Search Mode: This is triggered when the debouncedSearch is not empty. This calls searchSoftware() and will ignore the active filters object
2. Filter Mode: This is active when debouncedSearch is empty. This will then call the filterSoftware() with the active filters object. Any inactive filter keys are stripped before the query string is built.

Clearing the search input will automatically return to filter mode on the next debounce cycle.

9. Coverage Score Calculation

The coverage scores are computed on the frontend based on the feature flags returned by the API. The backend also has the precomputed coverage scores in the Firebase, however, the detail page recalculates them client side for live data accuracy.

10.1 Calculation Logic

The `computeOverallCoverage()` function iterates all keys across all four feature sections. This collects the nonnull values and returns the percentage that equal 'Y' and rounds to the nearest integer.

The `computeSectionScore()` uses the same logic applied to only the keys in a single feature section

Any null values are excluded from both the numerator and denominator when calculating. Therefore, a feature with no data doesn't count against the score.

10.2 Color Thresholds

Score Range	Color	Variable
75% – 100%	Green	--green
40% – 74%	Amber	--amber
0% – 39%	Red	--red

9. Local Development

11.1 Prerequisites

1. Node.js 18 or later
2. NPM 9 or later
3. The backend API code running locally (see backend documentation) or via the deployed URL

11.2 Installation

1. Clone the github repo into a folder
2. Run "npm install" to get the NPM dependencies

11.3 Environment Variables

1. Create a .env file in the root directory, the same place as the package.json
2. Put in the "REACT_APP_API_URL" environment variable

11.4 Running the Dev Server

1. Do npm start to begin the frontend page

10. Extending the Website

10.1 Adding a new Page

1. Create a new .jsx file in the /src/pages/ folder
2. Add the following code to your App.js
 - a. `<Route path="/pathname" element={<new page/>} />`
3. Add a `<Link>` tag in the navbar.jsx file it it needs to be in the navbar

10.2 Adding a new API function

1. Create the route in the backend
2. Add a named export to the /src/api/client.js file
3. Import and call it from any page or hook

10.3 Pushing the changes

Follow the instructions laid out in the deployment and management documentation to update the website with new changes.

11. SPA Redirect Rule

Since the app uses a client side routing with BrowserRouter, the direct URL access to the paths like /directory will result in a 404 from the server. Therefore, a render redirect rule can be setup to prevent this:

Source	Destination	Status	Purpose
/*	/index.html	200 , Rewrite	This will serve the react app for all the routes thus allowing the client side routing to handle the paths.